



Objective & Lecture Mapping: In previous labs, we built a Simple Reflex Agent that moved randomly or reacted only to immediate adjacent cells. Today, we transition to a **Goal-Based/Planning Agent**. You will expose the environment's state space to the agent, allowing it to use **Breadth-First Search (BFS)**, **Depth-First Search (DFS)**, and **Uniform-Cost Search (UCS)** to simulate and find paths to the food before taking a physical action.

Part 1: Practical Implementation — Building the Search Agent

Step 1.1: Exposing the World Model

Classical search algorithms require an abstract model of the environment to simulate future states. Currently, your agent is "blind" to the overall map.

1. Create a new Branch called Week_03 in your Forked Github Repo and work on the below Questions.
2. Open `visual_grid_game.py` .
3. Locate the `get_percept(self)` method.
4. Add three new keys to the dictionary to pass the global state to the agent:

```
'grid_size': (self.width, self.height)
'walls': list(self.walls)
'all_food': list(self.food_positions)
```

Step 1.2: Implementing BFS, DFS, and UCS

You will implement three distinct uninformed search strategies. The core node expansion structure is identical; only the data structure for the **Frontier** changes!

1. Open `agent.py` and create a new class called `SearchAgent` .
2. Import required structures: `from collections import deque` and `import heapq` .

3. **BFS:** Create `def bfs_search(...)` . Use a FIFO queue (`deque.popleft()`). This explores the shallowest nodes first.
4. **DFS:** Create `def dfs_search(...)` . Use a LIFO stack (`list.pop()`). This explores the deepest nodes first.
5. **UCS:** Create `def ucs_search(...)` . Use a Priority Queue (`heapq.heappop()`) ordered by the total path cost $g(n)$.
6. For all three algorithms, you must maintain a `reached` set (or visited array) to track explored states. This converts your algorithm from a *Tree Search* to a *Graph Search*, preventing infinite loops.

Step 1.3: Executing and Comparing Offline Plans

The agent must now form a complete plan and execute it step-by-step.

1. In your `SearchAgent.__init__` , add an empty list: `self.plan = []` and a config string `self.active_algo = 'BFS'` .
2. In `sense_and_act(self, percept)` , check if `self.plan` is empty.
3. If empty, find the closest food pellet from `percept['all_food']` , execute the search method matching `self.active_algo` , and store the resulting sequence of actions in `self.plan` .
4. Return the first action from the plan using `return self.plan.pop(0)` .
5. **Observation Task:** Change `self.active_algo` between 'BFS', 'DFS', and 'UCS' and run the simulation. Watch how DFS takes erratic, winding paths, while BFS and UCS take direct, optimal paths!

Part 2: Theoretical Evaluation

Based on your implementation and Lecture 04, complete the following analytical questions.

1. (Remember) You built your search logic based on the environment coordinates. What is the fundamental difference between the "State Space" and the "Search Tree"?

The **state space** is the complete set of all possible states that the agent can reach in the environment. In the grid, each valid coordinate represents a possible state. The **search tree** is the structure created by the search algorithm as it explores different paths from the initial state. Therefore, the state space represents the actual possible states, while the

search tree represents the paths explored to find a solution. The same state can appear multiple times in a search tree through different paths.

2. (Understand) In Step 1.2, you were instructed to use a `reached` set. In graph search theory, what specific problem does this set solve, and what would happen to your DFS agent without it?

The `reached` set keeps track of states that have already been visited. It prevents the search algorithm from repeatedly exploring the same states and entering cycles. Without it, DFS could repeatedly move between the same cells and potentially continue indefinitely without reaching the food. Therefore, the `reached` set prevents unnecessary exploration and converts the search from a **tree search into a graph search**.

3. (Analyze) Compare the visual paths generated by BFS and DFS in your simulation. Why is BFS guaranteed to be optimal in this specific grid, whereas DFS produces winding, suboptimal routes?

BFS explores nodes level by level, meaning it always explores the shallowest available states first. Since every movement in the grid has the same cost, the first path found by BFS is guaranteed to contain the minimum number of moves, making it optimal. DFS, however, explores one path as deeply as possible before backtracking. It may choose a path that moves away from the food and therefore produces a longer, winding route. Consequently, BFS is optimal for this grid, while DFS is not guaranteed to find the shortest path.

4. (Evaluate) If we scaled `visual\_grid\_game.py` up to a massive 1000x1000 grid, BFS and UCS might crash before finding food. According to the time and space complexity formulas from the lecture, contrast the memory bottlenecks of BFS versus DFS.

BFS has a time complexity of approximately $O(b^d)$ and a space complexity of $O(b^d)$, where b is the branching factor and d is the depth of the shallowest solution. Its main problem is memory because it must store a large frontier of unexplored nodes.

DFS has a time complexity of approximately $O(b^m)$ and a space complexity of $O(bm)$, where m is the maximum search depth. Therefore, DFS requires much less memory than BFS. On a 1000x1000 grid, BFS may run out of memory because of its large frontier, while DFS can use considerably less memory but may take much longer to find the food and does not guarantee an optimal path.

Once Completed Get a PDF of the completed File and Push into the Week 03 brach in the Github Project

Incomplete: 0/11 questions answered. Please provide detailed responses for all questions.



FACULTY OF COMPUTING